Design Pattern

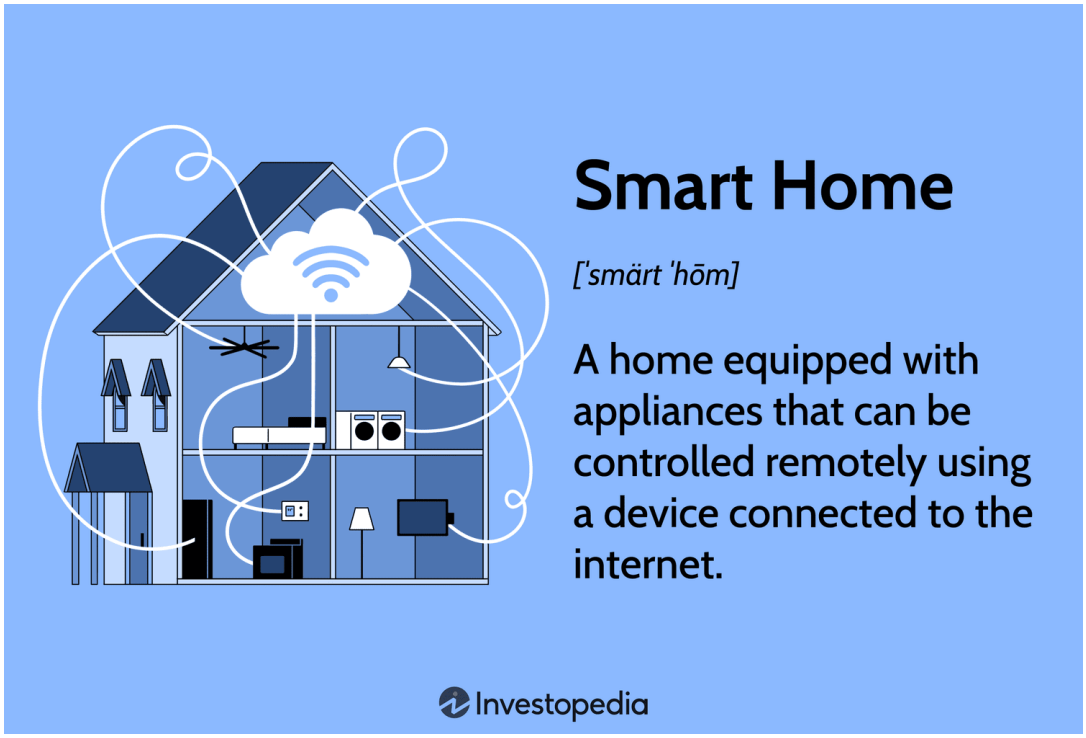
System Overview: A Smart Home Automation System is an integrated environment where multiple devices, services, and control mechanisms work together to automate and manage household activities such as security, lighting, temperature, and entertainment.

The system is designed around three key ideas:

Interoperability → different devices working together

Centralized control → one system manages everything

Automation & intelligence → actions happen automatically or with minimal input



Source Code for all patterns:

git clone https://github.com/nahaQUT/design_patterns.git

1. Smart Home Automation – Adapter Design Pattern Documentation

This implementation showcases how the Adapter Pattern can bring legacy components into the fold of modern architectures, ensuring smooth functionality across different systems.

Step 1: Understanding Adapter Design Pattern:

Introduction

This Python program demonstrates the Adapter Design Pattern by enabling the integration of a legacy home security system into a modern smart home ecosystem. The Adapter bridges the gap between the incompatible interfaces of the old system and the new smart home controller, allowing seamless operation.

Adapter Design Pattern Concept

The Adapter Pattern allows objects with incompatible interfaces to work together. It acts as a bridge that converts the interface of one class into another that the client expects.

How It Works in This Code

The LegacySecurityAdapter class acts as the Adapter that enables compatibility between the LegacySecuritySystem and the SmartHomeController.

- **LegacySecuritySystem**: The old security system with its own interface.
- **SmartHomeController**: The unified smart home controller with a standard interface.

The client interacts with the SmartHomeController, which internally uses the Adapter to integrate the LegacySecuritySystem.

Benefits of Using the Adapter Pattern

- **Compatibility**: Integrates old components into new systems without modifying their source code.
- **Flexibility**: Facilitates reuse of legacy components in different contexts.
- **Seamless Integration**: Provides a unified interface for clients.
- **Maintainability**: Changes to the adapter do not affect the client or the subsystem.

Step 2: Code Structure

Subsystem Class (Legacy Component)

The LegacySecuritySystem class represents the outdated security system with its own methods.

Example:

Legacy Component 1:

```
class LegacySecuritySystem:
    def arm_legacy(self):
        print("Legacy Security System Armed")

    def disarm_legacy(self):
        print("Legacy Security System Disarmed")

    def trigger_alarm_legacy(self):
        print("Legacy Security Alarm Triggered!")
```

Legacy Component 2:

```
class LegacySecuritySystem2:
    def arm_legacy2(self):
        print("Legacy Security System Armed 2")

    def disarm_legacy2(self):
        print("Legacy Security System Disarmed 2")

    def trigger_alarm_legacy2(self):
        print("Legacy Security Alarm Triggered! 2")
```

Adapter Class (Bridge)

The LegacySecurityAdapter class adapts the interface of the LegacySecuritySystem to match the expected interface of the SmartHomeController. Example:

```
class LegacySecurityAdapter:
    def __init__(self, legacy_system):
        self.legacy_system = legacy_system

    def arm(self):
        self.legacy_system.arm_legacy()

    def disarm(self):
        self.legacy_system.disarm_legacy()

    def trigger_alarm(self):
        self.legacy_system.trigger_alarm_legacy()
```

Smart Home Controller (Client)

The SmartHomeController class interacts with devices through a standard interface, including the adapted legacy security system. Example:

```
class SmartHomeController:
    def __init__(self):
        self.security_system = None

    def integrate_security_system(self, security_system):
        self.security_system = security_system

    def activate_security(self):
        print("Activating Security System...")
        self.security_system.arm()

    def deactivate_security(self):
        print("Deactivating Security System...")
        self.security_system.disarm()

    def alert_security_breach(self):
        print("Security Breach Detected!")
        self.security_system.trigger_alarm()
```

Example Usage in Main functions:

```
if __name__ == "__main__":
    legacy_system = LegacySecuritySystem()
    adapter = LegacySecurityAdapter(legacy_system)
```

```
smart_controller = SmartHomeController()
smart_controller.integrate_security_system(adapter)

smart_controller.activate_security()
smart_controller.alert_security_breach()
smart_controller.deactivate_security()
```

2. Smart Home Automation – Singleton Design Pattern Documentation

This implementation illustrates how the Singleton Pattern can provide a unified and efficient approach to managing shared resources in a smart home ecosystem.

Step 1: Understanding Singleton Design:

Introduction

This Python program demonstrates the Singleton Design Pattern by implementing a centralized Smart Home Configuration Manager. The Singleton ensures that only one instance of the configuration manager exists, providing a global point of access to shared settings for various smart home devices.

Singleton Design Pattern Concept

The Singleton Pattern ensures that a class has only one instance and provides a global access point to that instance. This is particularly useful when managing shared resources or ensuring a single point of coordination.

How It Works in the Solution Code

The `SmartHomeConfigManager` class is implemented as a Singleton, ensuring that all components of the smart home system share the same configuration settings. The Singleton uses a private constructor and a static method to control the instantiation process.

Benefits of Using the Singleton Pattern

- **Global Access Point**: Provides a single, well-defined access point for shared resources.
- **Controlled Instantiation**: Prevents multiple instances of a class, ensuring consistency.
- **Resource Efficiency**: Reduces overhead by reusing a single instance.
- **Ease of Coordination**: Simplifies management of shared state or resources.

Step 2: Code Structure

Singleton Class (`SmartHomeConfigManager`)

The `SmartHomeConfigManager` class ensures only one instance exists and provides access to shared configurations for all devices.

```
class SmartHomeConfigManager:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if not cls._instance:
            cls._instance = super(SmartHomeConfigManager, cls).__new__(cls, *args,
**kwargs)
            cls._instance.settings = {}
        return cls._instance

    def set_setting(self, key, value):
        self.settings[key] = value

    def get_setting(self, key):
        return self.settings.get(key, None)
```

Example Components Using the Singleton:

Devices interact with the `SmartHomeConfigManager` to retrieve or update global settings.

```
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.config = SmartHomeConfigManager()
```

```
    def display_settings(self):
        print(f"[{self.name}] Current Settings: {self.config.settings}")

    def update_setting(self, key, value):
        print(f"[{self.name}] Updating {key} to {value}")
        self.config.set_setting(key, value)
```

Example Usage in the main:

```
if __name__ == "__main__":
    # Retrieve the Singleton instance
    config_manager = SmartHomeConfigManager()

    # Set initial configuration
    config_manager.set_setting("temperature", 22)
    config_manager.set_setting("lights", "dim")

    # Create smart devices
    device1 = SmartDevice("Thermostat")
    device2 = SmartDevice("Living Room Lights")

    # Display shared settings
    device1.display_settings()
    device2.display_settings()

    # Update settings from one device
    device1.update_setting("temperature", 24)

    # Display updated settings
    device1.display_settings()
    device2.display_settings()
```

3. **Smart Home Automation – Facade Design Pattern Documentation**

Step 1: Understanding Facade Design Pattern:

Introduction

This Python program demonstrates the Facade Design Pattern by simplifying the control of multiple smart home devices (TV, Sound System, Lights, and Air Conditioner) through a unified interface. Instead of interacting with each device individually, users can activate a "Movie Night Mode" or shut everything down with a single command.

Facade Design Pattern Concept

The Facade Pattern provides a simplified interface to a complex subsystem. It hides the complexity of individual components and exposes only the necessary functionality.

How It Works in This Code

- The *SmartHomeFacade* class acts as the Facade that manages multiple subsystems
  - TV (TV class)
  - Sound System (*SoundSystem* class)
  - Lights (Lights class)
  - Air Conditioner (*AirConditioner* class)
- The user interacts only with the Facade, without dealing with each subsystem separately.
- The Facade internally delegates requests to the appropriate subsystem.

Benefits of Using the Facade Pattern

- Simplifies Complexity – Users don't need to interact with multiple devices separately.
- Improves Code Maintainability – Changes in subsystems don't affect the client.

- ❖ Encapsulation – Internal workings of subsystems are hidden from users.
- ❖ Extensibility – More devices (e.g., Smart Curtains, Security System) can be added without modifying client code.

Step 2: Code Structure

Subsystem Classes (Complex Components)

Each device (TV, Sound System, Lights, and AC) is represented as a separate class, with methods to turn on/off and adjust settings. Example:

```
class TV:
    def on(self):
        print("TV is now ON")

    def off(self):
        print("TV is now OFF")

    def set_channel(self, channel):
        print(f"TV is set to channel {channel}")
```

*** Follow Github for rest of the code

The Facade Class (Simplified Interface)

The SmartHomeFacade class provides an easy-to-use interface, wrapping multiple device interactions into simple methods. Example:

```
class SmartHomeFacade:
    def __init__(self):
        self.tv = TV()
        self.sound = SoundSystem()
        self.lights = Lights()
        self.ac = AirConditioner()
```

```
    def start_movie_night(self):
        print("\nStarting Movie Night Mode...")
        self.tv.on()
        self.tv.set_channel("Netflix")
        self.sound.on()
        self.sound.set_volume(20)
        self.lights.dim()
        self.ac.on()
        self.ac.set_temperature(22)
        print("Enjoy your movie! \n")

    def shutdown(self):
        print("\nShutting down all systems...")
        self.tv.off()
        self.sound.off()
        self.lights.bright()
        self.ac.off()
        print("All devices are turned off. Goodbye!\n")
```

Call the Facade class

```
obj = SmartHomeFacade()

obj.start_movie_night()

obj.shutdown()
```

4. Factory Design Pattern Documentation

This implementation showcases how the Factory Pattern can simplify object creation and improve the modularity of your code.

Step 1: Understanding Factory Design Pattern:

Introduction

This Python program demonstrates the Factory Design Pattern by implementing a simple vehicle creation system. The Factory Pattern is used to create objects without specifying their exact class, providing flexibility and scalability.

Factory Design Pattern Concept

The Factory Pattern defines an interface for creating objects but allows subclasses or methods to decide which class to instantiate. It encapsulates object creation logic, promoting code reusability and flexibility.

How It Works in This Code

The `VehicleFactory` class acts as the Factory, responsible for creating instances of different vehicle types based on a provided input. Each vehicle type (Car, Bike, Truck) is represented by a separate class.

Benefits of Using the Factory Pattern

- **Encapsulation**: Hides the object creation logic, making the code easier to maintain.
- **Flexibility**: Allows the addition of new object types without modifying the existing factory.
- **Reusability**: Centralizes object creation in one location, reducing redundancy.
- **Scalability**: Supports the creation of a wide range of related objects dynamically.

Step 2: Code Structure

Product Classes (Vehicles)

Each type of vehicle (Car, Bike, Truck) is represented by a class with its own behavior. Example:

```
class Car:
    def drive(self):
        print("Driving a car.")

class Bike:
    def drive(self):
        print("Riding a bike.")

class Truck:
    def drive(self):
        print("Driving a truck.")
```

Factory Class

The `VehicleFactory` class encapsulates the creation logic, returning instances of the appropriate vehicle type based on input. Example:

```
class VehicleFactory:
    @staticmethod
    def get_vehicle(vehicle_type):
        if vehicle_type == "car":
            return Car()
        elif vehicle_type == "bike":
            return Bike()
        elif vehicle_type == "truck":
            return Truck()
        else:
            raise ValueError("Unknown vehicle type")
```

Client Code

The client interacts with the Factory to create and use vehicle instances without worrying about their specific implementation. Example:

```
if __name__ == "__main__":
    factory = VehicleFactory()

    # Create a car
    car = factory.get_vehicle("car")
    car.drive()

    # Create a bike
    bike = factory.get_vehicle("bike")
    bike.drive()

    # Create a truck
    truck = factory.get_vehicle("truck")
    truck.drive()
```

Activity 1: Now Create a Smart Home Device Factory where instead of “car, bike, truck”, you will use “light, fan, air conditioner”.

5. Middleware Design Pattern Documentation

This implementation demonstrates how the Middleware Pattern can streamline request processing in a modular and extensible way.

Step 1: Understanding Middleware Design Pattern:

Introduction

This Python program demonstrates the Middleware Pattern by implementing a request processing pipeline for a web application. Middleware components handle tasks such as authentication, logging, and input validation in a modular and chainable way.

Middleware Pattern Concept

The Middleware Pattern allows request processing to be divided into a chain of responsibility, where each middleware component performs a specific task and passes the request to the next component in the chain. This promotes flexibility, reusability, and separation of concerns.

How It Works in This Code

The `Middleware` class defines a base structure for middleware components. Each middleware class performs a task and passes the request to the next middleware in the chain. The `MiddlewareManager` sets up the chain and processes the request.

Benefits of Using the Middleware Pattern

- **Separation of Concerns**: Each middleware component handles a specific task, making the code modular and maintainable.
- **Flexibility**: Middleware can be added, removed, or reordered without affecting the overall structure.
- **Reusability**: Middleware components can be reused across different applications or projects.
- **Scalability**: New functionality can be added easily by introducing additional middleware components.

Step 2: Code Structure

Base Middleware Class

The `Middleware` class provides a structure for middleware components, ensuring consistency. Example:

```
class Middleware:
    def __init__(self, next_middleware=None):
        self.next_middleware = next_middleware

    def process(self, request):
        if self.next_middleware:
            self.next_middleware.process(request)
```

Concrete Middleware Components

Each middleware component performs a specific task. Example:

```
class AuthenticationMiddleware(Middleware):
    def process(self, request):
        if "user" in request:
            print("Authentication successful.")
        else:
            print("Authentication failed.")
        super().process(request)

class LoggingMiddleware(Middleware):
    def process(self, request):
        print(f"Logging request: {request}")
        super().process(request)

class ValidationMiddleware(Middleware):
    def process(self, request):
        if "data" in request:
            print("Request data validated.")
        else:
            print("Invalid request data.")
        super().process(request)
```

Middleware Manager

The `MiddlewareManager` sets up the chain of middleware and initiates request processing. Example:

```
class MiddlewareManager:
    def __init__(self):
        self.chain = None

    def add_middleware(self, middleware):
        if not self.chain:
            self.chain = middleware
        else:
            current = self.chain
            while current.next_middleware:
                current = current.next_middleware
            current.next_middleware = middleware

    def process_request(self, request):
        if self.chain:
            self.chain.process(request)
```

Example Usage in Main

```
if __name__ == "__main__":
    manager = MiddlewareManager()

    # Add middleware components to the chain
    manager.add_middleware(AuthenticationMiddleware())
    manager.add_middleware(LoggingMiddleware())
    manager.add_middleware(ValidationMiddleware())

    # Create a request object
    request = {"user": "Alice", "data": "Sample data"}

    # Process the request through the middleware chain
    manager.process_request(request)
```

*** Think of this authentication, logging for smart home system.

Link for study help (you will find codes in various languages, description and examples of the patterns): https://refactoring.guru/